

Item 06 – LLM Observability & Debugging

Pareto Learning Guide (2026)

A Hiring-Oriented Curriculum by Expert Panel

1. Executive Summary

Production LLM systems are black boxes. Without observability, you're flying blind. This module teaches structured logging, metrics, tracing, and systematic debugging for LLM applications.



AI Product Engineer: *Observability is the difference between "it works on my machine" and "I can debug production failures." Senior engineers instrument everything. Juniors debug by guessing.*

- • Structured logging for LLM inputs/outputs
- • Metrics: latency, cost, quality, error rates
- • Distributed tracing for multi-step LLM workflows
- • Debugging frameworks for production failures
- • Portfolio: Monitored system with dashboards

2. The Critical 20%

Concept	Why Critical	Deferred
Structured Logging	Record everything for post-mortem analysis	Advanced log aggregation systems
Latency Tracking	P50/P95/P99 metrics reveal bottlenecks	Statistical process control
Cost Monitoring	Prevent budget overruns	Advanced FinOps practices
Error Classification	Systematic debugging starts with categorization	Formal error taxonomies
Distributed Tracing	Debug multi-step workflows	OpenTelemetry deep dive

3. Engineering-First Deep Understanding

3.1 Structured Logging for LLMs

Every LLM call should log: timestamp, prompt, response, model, tokens, cost, latency, user_id, session_id.

```
import logging
import json

def log_llm_call(prompt, response, metadata):
    log_entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "prompt_hash": hash(prompt),
        "response_hash": hash(response),
        "model": metadata["model"],
        "input_tokens": metadata["input_tokens"],
        "output_tokens": metadata["output_tokens"],
        "cost": calculate_cost(metadata),
        "latency_ms": metadata["latency_ms"],
        "user_id": metadata["user_id"],
        "error": metadata.get("error", None)
    }
    logging.info(json.dumps(log_entry))
```

💡 **Applied AI Engineer:** Log structured JSON, not text. JSON is queryable, aggregatable, and indexable. Text logs are useless at scale.

3.2 Metrics That Matter

- Track these metrics in production:
- • Latency: P50, P95, P99 (median, 95th percentile, 99th percentile)
- • Cost: \$/request, daily burn rate, monthly projection
- • Quality: error rate, user satisfaction, accuracy on test set
- • Volume: requests/day, tokens/day, cache hit rate

4. Day-by-Day Skill Reconstruction Plan

12-day intensive on observability and debugging.

- Day 1: Structured Logging Setup
- Day 2: Metrics Collection (latency, cost, quality)
- Day 3: Dashboard Creation (Grafana/Datadog)
- Day 4: Alerting System (cost spikes, latency degradation)
- Day 5-6: Project - Instrumented RAG system with dashboard
- Day 7: Distributed Tracing (multi-step workflows)
- Day 8: Error Classification and Analysis
- Day 9: Debugging Framework (systematic root cause analysis)
- Day 10-12: Capstone - Production monitoring system

5. Common Blind Spots

- • Not logging prompts (privacy-safe hashes)
- • Only tracking averages (P95/P99 reveal issues)
- • No cost alerts (runaway costs)
- • Ignoring error patterns (systematic failures)

6. Self-Assessment Questions

1. Q1: What metrics would you track for a RAG system?
2. Q2: Your P99 latency is 10s but P50 is 1s. Diagnosis?
3. Q3: Design an alerting system for cost overruns.
4. Q4: How do you debug a 5% error rate spike?

Expert Panel Synthesis

Dimension	Score	Evidence
Hiring Relevance	0.85	Observability is critical for production systems
Practical Applicability	0.90	Immediate improvements to system reliability